



Sichuan University -
Pittsburgh Institute

ECEo202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute



Topics

- Interrupt Number
- Enable/Disable
- Priority and Preemption
- Mask

Interrupt Number

- Cortex-M supports up to 256 interrupts.



Cortex Microcontroller Software Interface Standard (CMSIS)

Interrupt Number

- First 16 are system exceptions
 - CMSIS defines their interrupt numbers as negative
 - Defined by ARM core
- The rest 240 are peripheral interrupts
 - Peripheral interrupt number starts with 0.
 - Defined by chip manufacturers.



stm321476xx.h

```

/***** Cortex-M4 System Exceptions *****/

NonMaskableInt_IRQn    = -14,    /* 2 Cortex-M4 Non Maskable Interrupt */
HardFault_IRQn         = -13,    /* 3 Cortex-M4 Hard Fault Interrupt */
MemoryManagement_IRQn  = -12,    /* 4 Cortex-M4 Memory Management Interrupt */
BusFault_IRQn          = -11,    /* 5 Cortex-M4 Bus Fault Interrupt */
UsageFault_IRQn        = -10,    /* 6 Cortex-M4 Usage Fault Interrupt */
SVCall_IRQn            = -5,     /* 11 Cortex-M4 SV Call Interrupt */
DebugMonitor_IRQn      = -4,     /* 12 Cortex-M4 Debug Monitor Interrupt */
PendSV_IRQn            = -2,     /* 14 Cortex-M4 Pend SV Interrupt */
SysTick_IRQn           = -1,     /* 15 Cortex-M4 System Tick Interrupt */

/***** Peripheral Interrupt Numbers *****/

WWDG_IRQn              = 0,      /* Window WatchDog Interrupt */
PVD_PVM_IRQn           = 1,      /* PVD/PVM1,2,3,4 through EXTI Line detection Interrupts */
TAMP_STAMP_IRQn        = 2,      /* Tamper and TimeStamp interrupts through the EXTI line */
RTC_WKUP_IRQn          = 3,      /* RTC Wakeup interrupt through the EXTI line */
FLASH_IRQn             = 4,      /* FLASH global Interrupt */
RCC_IRQn               = 5,      /* RCC global Interrupt */
EXTI0_IRQn             = 6,      /* EXTI Line0 Interrupt

```

**System
Exceptions**

**Peripheral
Interrupts**

Enable an Interrupt

- Enable a system exception
 - Some are always enabled (cannot be disabled)
 - No centralized registers for enabling/disabling
 - Each are control by its corresponding components, such as SysTick module
- Enable a peripheral interrupt
 - Centralized register arrays for enabling/disabling
 - **ISER** registers for enabling
 - **ICER** registers for disabling

Enabling Peripheral Interrupts

Interrupt Set Enable Register 0 (ISER0)

STM 32 L1

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Enable Bit	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Interrupt Number	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	I2C1_EV	TIM4	TIM3	TIM2	TIM11	TIM10	TIM9	LCD	EXTI9_5	COMP	DAC	USB_LP	USB_HP	ADC1	DMA1_CH7	DMA1_CH6	DMA1_CH5	DMA1_CH4	DMA1_CH3	DMA1_CH2	DMA1_CH1	EXTI4	EXTI3	EXTI2	EXTI1	EXTI0	RCC	FLASH	RTC_WKUP	TAMPER_STAMP	PVD	WWDG

Interrupt Set Enable Register 1 (ISER1)

Address of ISER1 = Address of ISER0 + 4

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Enable Bit	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
Interrupt Number																			44	43	42	41	40	39	38	37	36	35	34	33	32
																			TIM7	TIM6	USB_FS_WKUP	RTC_Alarm	EXTI15_10	USART3	USART2	USART1	SPI2	SPI1	I2C2_ER	I2C2_EV	I2C1_ER

```
stm32l476xx_constants.s:
TIM7_IRQn EQU 55
```

```
LDR R0, =TIM7_IRQn
BL NVIC_EnableIRQ ;Enable Timer7 interrupt
```


Disabling Peripheral Interrupts

Interrupt Clear Enable Register 0 (ICER0)

STM 32 L1

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Clear Enable Bit	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Interrupt Number	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	I2C1_EV	TIM4	TIM3	TIM2	TIM11	TIM10	TIM9	LCD	EXTI9_5	COMP	DAC	USB_LP	USB_HP	ADC1	DMA1_CH7	DMA1_CH6	DMA1_CH5	DMA1_CH4	DMA1_CH3	DMA1_CH2	DMA1_CH1	EXTI4	EXTI3	EXTI2	EXTI1	EXTI0	RCC	FLASH	RTC_WKUP	TAMPER_STAMP	PVD	WWDG

Interrupt Clear Enable Register 1 (ICER1) *Address of ICER1 = Address of ICER0 + 4*

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Clear Enable Bit	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
Interrupt Number																				44	43	42	41	40	39	38	37	36	35	34	33	32
																				TIM7	TIM6	USB_FS_WKUP	RTC_Alarm	EXTI15_10	USART3	USART2	USART1	SPI2	SPI1	I2C2_ER	I2C2_EV	I2C1_ER

```
stm32l476xx_constants.s:
TIM7_IRQn EQU 55
```

```
LDR R0, =TIM7_IRQn
BL NVIC_DisableIRQ ;Disable Timer7 interrupt
```

Interrupt Priority

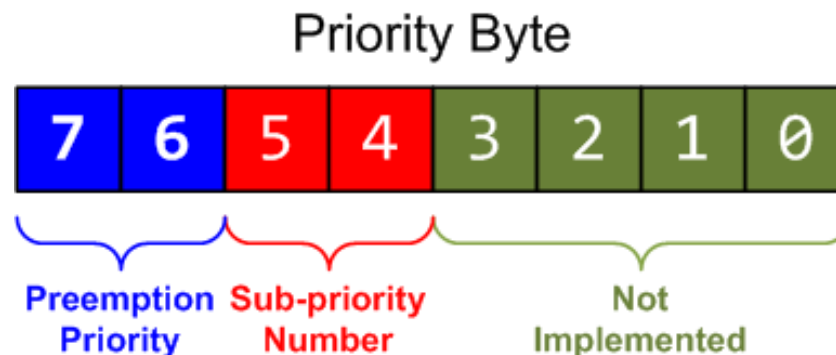
- Inverse Relationship:
 - Lower priority value means higher urgency.
 - Priority of Interrupt A = 5,
 - Priority of Interrupt B = 2,
 - B has a higher priority/urgency than A.
- Fixed priority for Reset, HardFault, and NMI.

Exception	IRQn	Priority
Reset	N/A	-3 (the highest)
Non-maskable Interrupt (NMI)	-14	-2 (2 nd highest)
Hard Fault	-13	-1

- Adjustable for all the other interrupts

Interrupt Priority

- Interrupt priority is configured by **Interrupt Priority Register (IP)**
- Each priority consists of two fields, including **preempt priority number** and **sub-priority number**.
 - The preempt priority number defines the priority for preemption.
 - The sub-priority number determines the order when multiple interrupts are pending with the same preempt priority number.

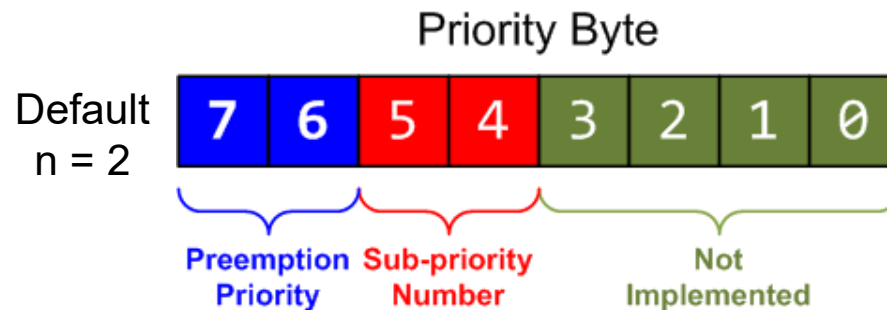


default setting

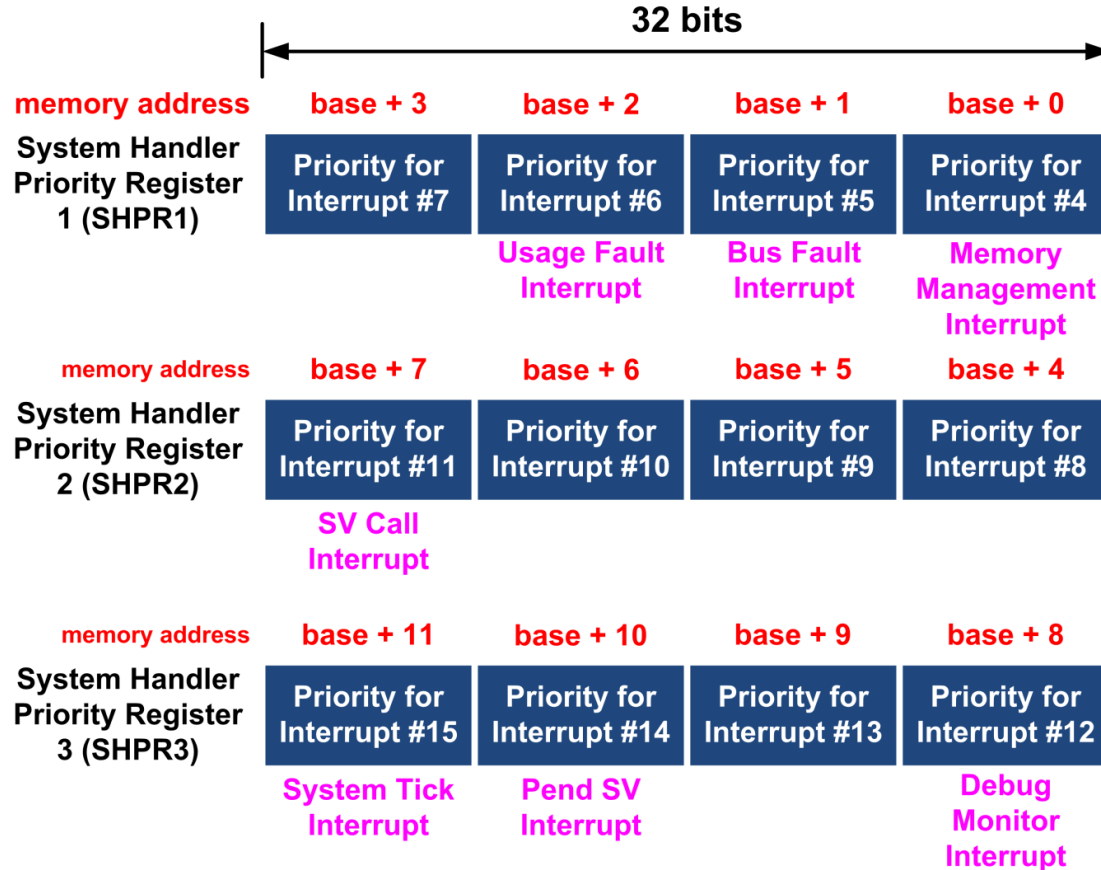
Preemption and Sub-priority Configuration

- NVIC_SetPriorityGrouping(n)
 - Perform unlock, and update AIRCR register

n	# of bits in preemption priority	# of bits in sub- priority
0	0	4
1	1	3
2 (default)	2	2
3	3	1
4	4	0



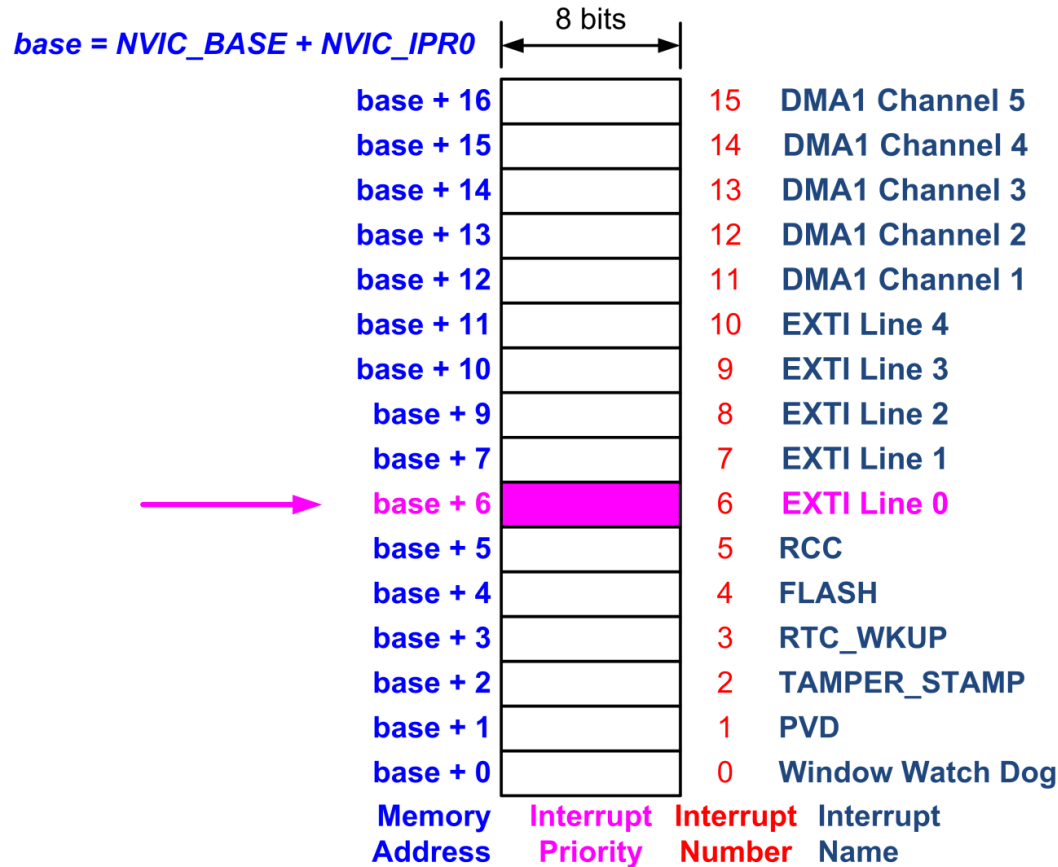
Priority of System Interrupts



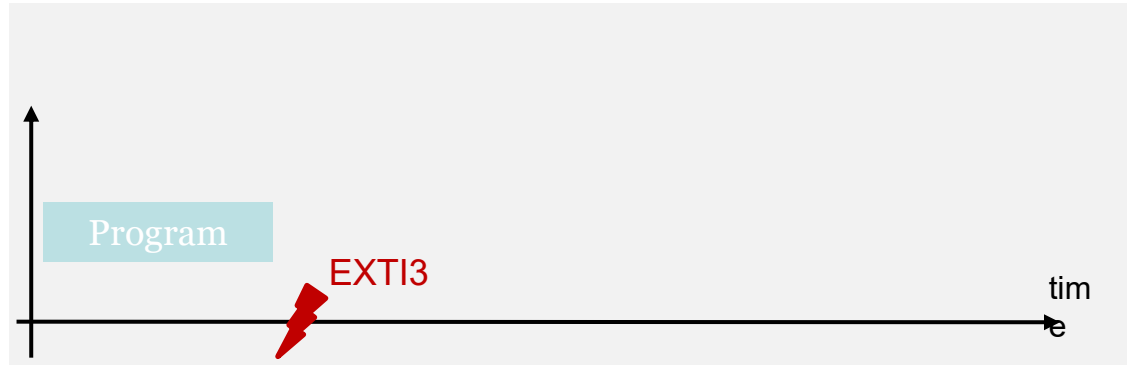
```

LDR R0, =TIM7_IRQn           ;which interrupt
MOV R1, #0                   ;priority
BL NVIC_SetPriority           ;set priority level
    
```

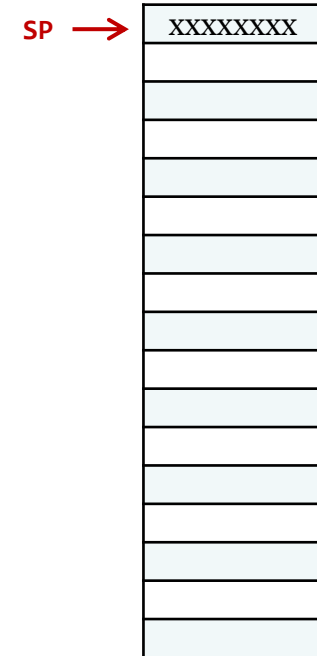
Priority of Peripheral Interrupts



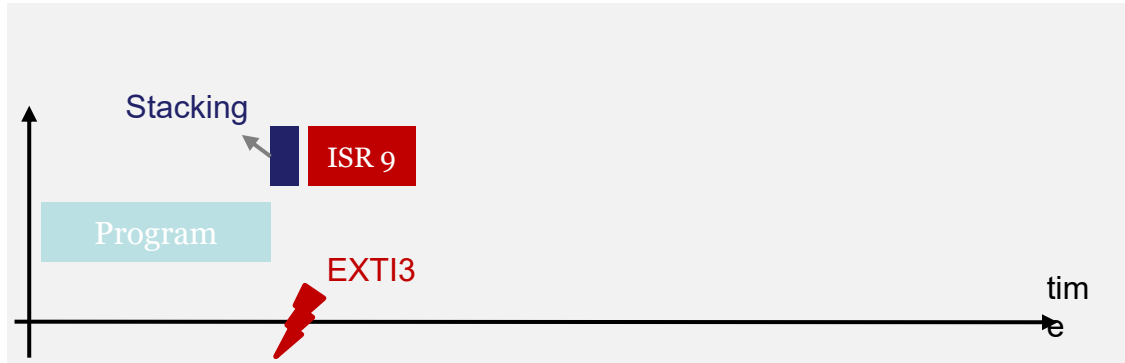
Nested Interrupts: Example of Preemption



	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3



Nested Interrupts: Example of Preemption

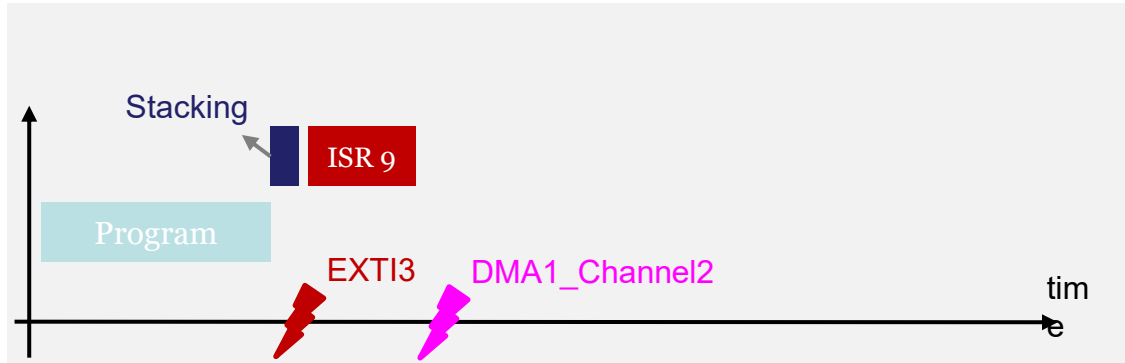


	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3

SP →

XXXXXXXX
xPSR
PC(r15)
LR(r14)
r12
r3
r2
r1
r0

Nested Interrupts: Example of Preemption



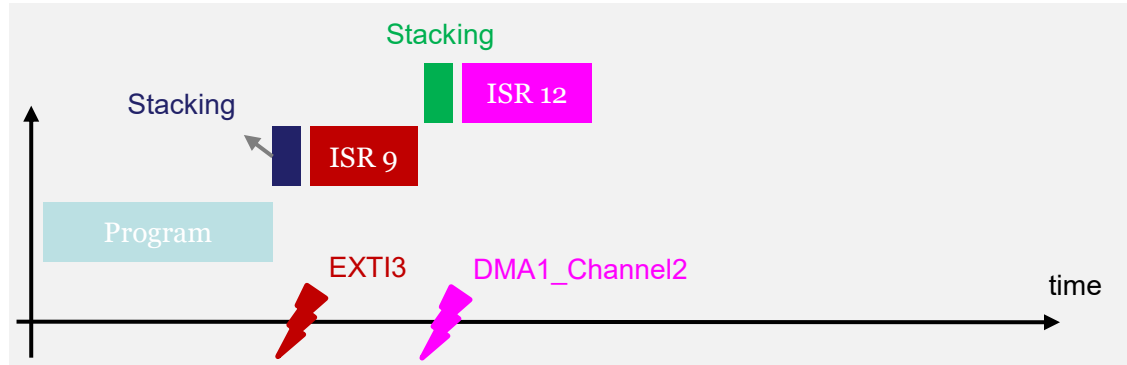
	DMA1_Channel2			EXTIS3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	1	0	0	0	0
Priority Register	3	4	7	5	3

Lower priority value means higher urgency.

SP →

XXXXXXXX
xPSR
PC(r15)
LR(r14)
r12
r3
r2
r1
r0

Nested Interrupts: Example of Preemption

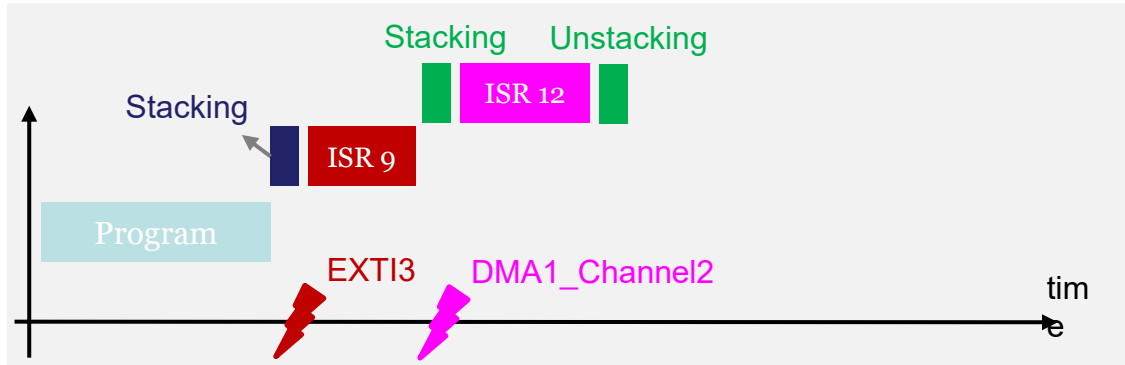


	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	1	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3

XXXXXXXX
xPSR
PC(r15)
LR(r14)
r12
r3
r2
r1
r0
xPSR
PC(r15)
LR(r14)
r12
r3
r2
r1
r0

SP →

Nested Interrupts: Example of Preemption

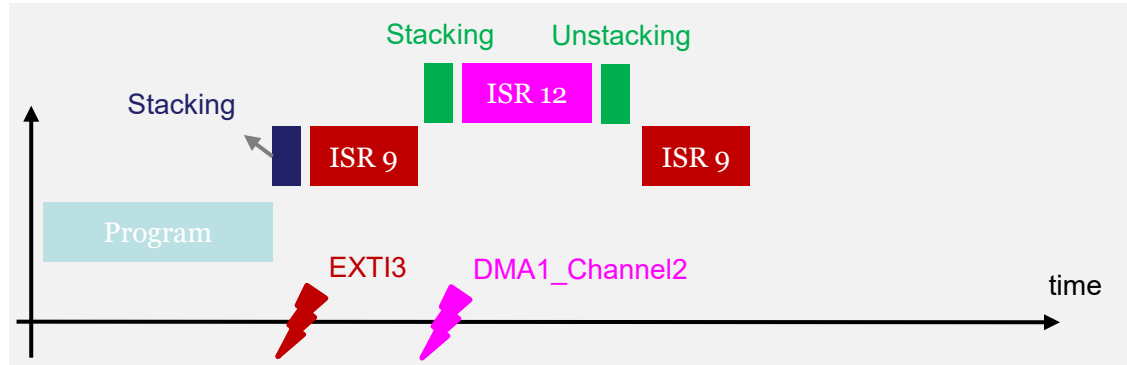


	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3

SP →

XXXXXXXX
xPSR
PC(r15)
LR(r14)
r12
r3
r2
r1
r0

Nested Interrupts: Example of Preemption

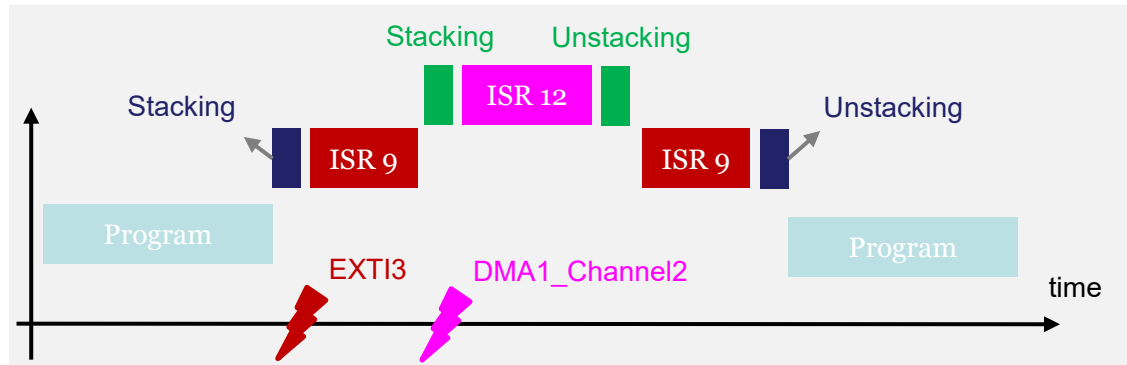


	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3

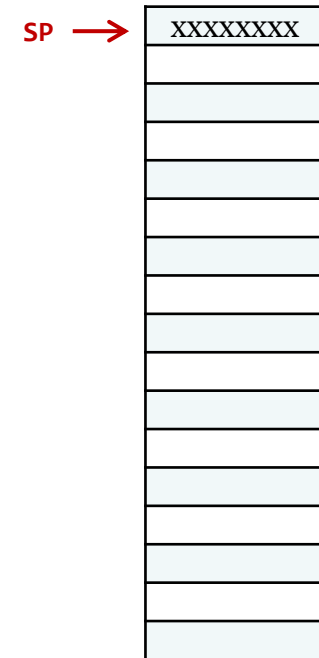
SP →

XXXXXXXX
xPSR
PC(r15)
LR(r14)
r12
r3
r2
r1
r0

Nested Interrupts: Example of Preemption

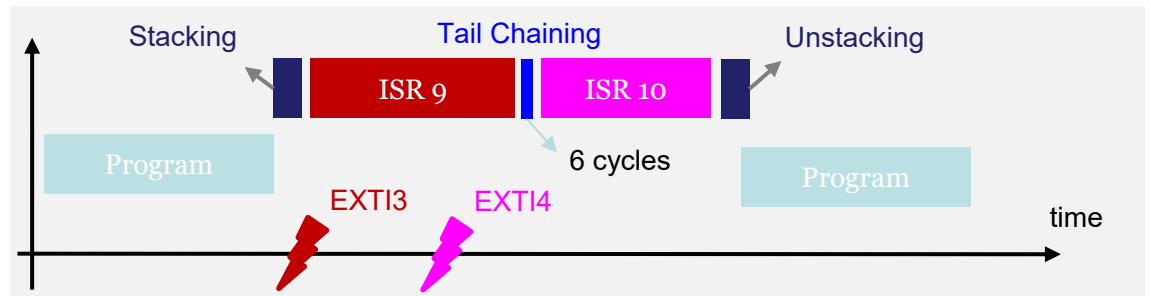
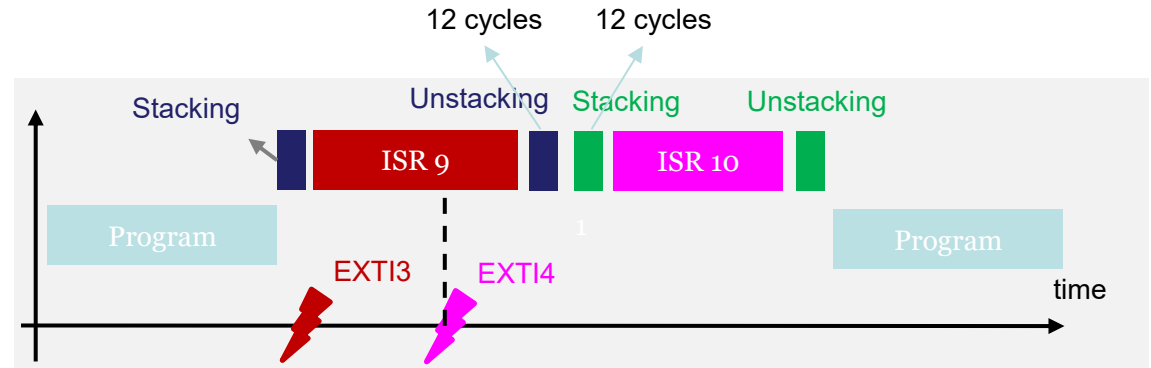


	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3



Nested Interrupts: Tail Chaining

- EXTI3 → ISR 9
- EXTI4 → ISR 10
- Suppose EXTI4 has less urgency than EXTI3.
 - EXTI4 has a higher numeric priority value than EXTI3.



Masking Priority

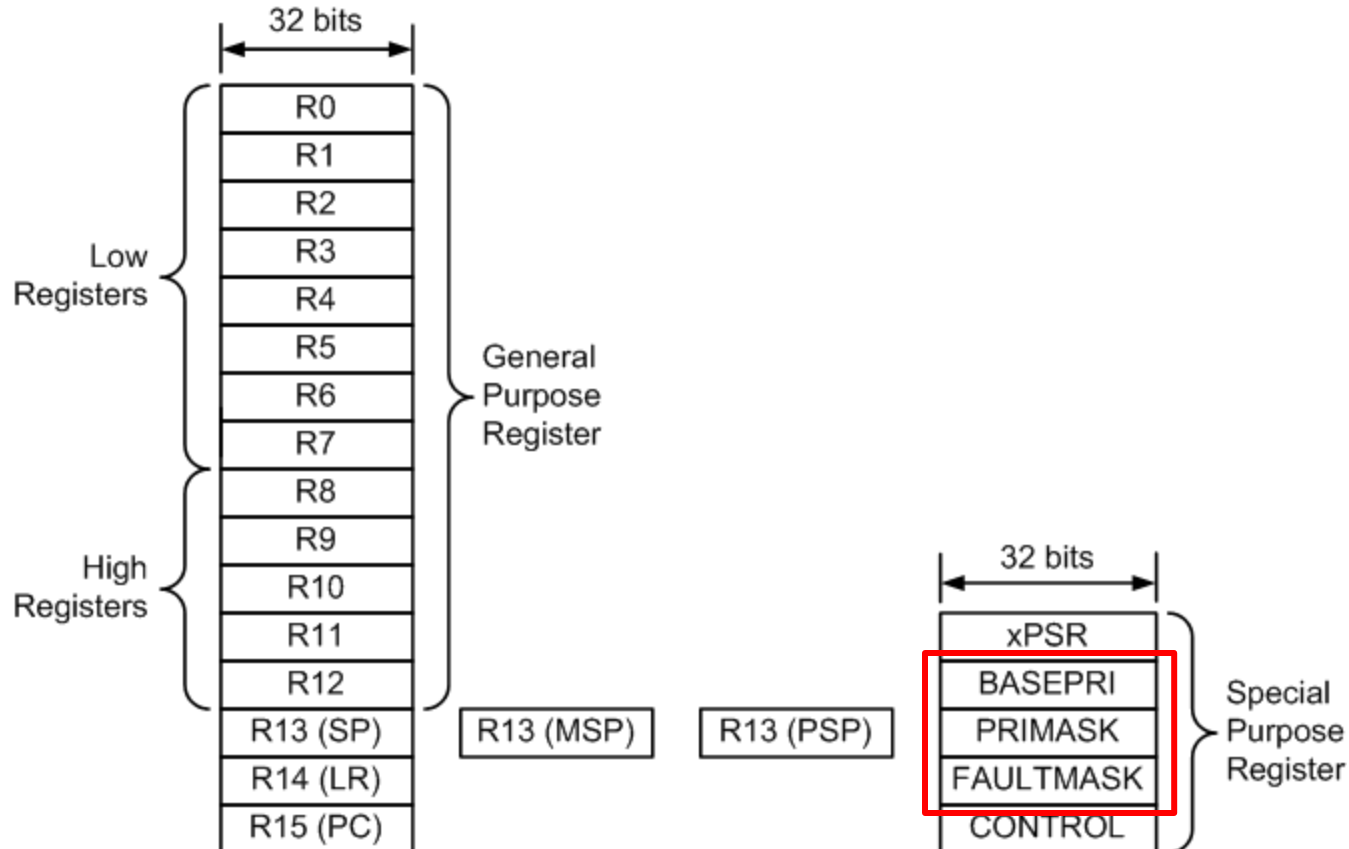
- Disable all interrupts with less urgency
 - For critical code, only certain interrupts are allowed.

Disable interrupts with priority value same or higher

```
// Critical code  
{  
...  
}
```

Re-enable all interrupts

Registers



Interrupt Priority

- Inverse Relationship:
 - Lower priority value means higher urgency.
 - Priority of Interrupt A = 5,
 - Priority of Interrupt B = 2,
 - B has a higher priority/urgency than A.
- Fixed priority for Reset, HardFault, and NMI.

Exception	IRQn	Priority
Reset	N/A	-3 (the highest)
Non-maskable Interrupt (NMI)	-14	-2 (2 nd highest)
Hard Fault	-13	-1

- Adjustable for all the other interrupts

Exception-masking registers (PRIMASK, FAULTMASK and BASEPRI)

- **FAULTMASK**: Used to disable all exceptions Non-maskable interrupt (NMI) .

- Write 1 to **FAULTMASK** to disable all interrupts except NMI

```
MOV R0, #1  
MSR FAULTMASK, R0
```

- Write 0 to **FAULTMASK** to enable all interrupts

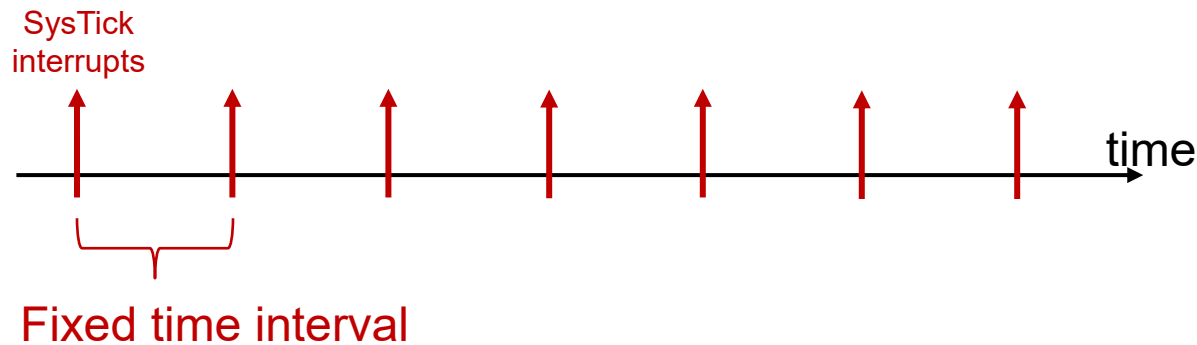
```
MOV R0, #0  
MSR FAULTMASK, R0
```

- **PRIMASK**: Used to disable all exceptions except NMI and hard fault.
- **BASEPRI**: Disable interrupts only with priority lower than or equal to a certain level
 - Example, disable all exceptions with priority level higher than or equal to 0x60

```
MOV R0, #0x60  
MSR BASEPRI, R0
```

System Timer (SysTick)

- Generate **SysTick interrupts** at a fixed time interval



- Example Usages:
 - Measuring time elapsed, such as time delay function
 - Executing tasks periodically, such as periodic polling, and OS CPU scheduling

System Timer (SysTick)

- System timer is a **standard** hardware component built into ARM Cortex-M.
- This hardware **periodically** forces the processor to execute the following ISR:

SysTick_Handler PROC

BX LR
ENDP

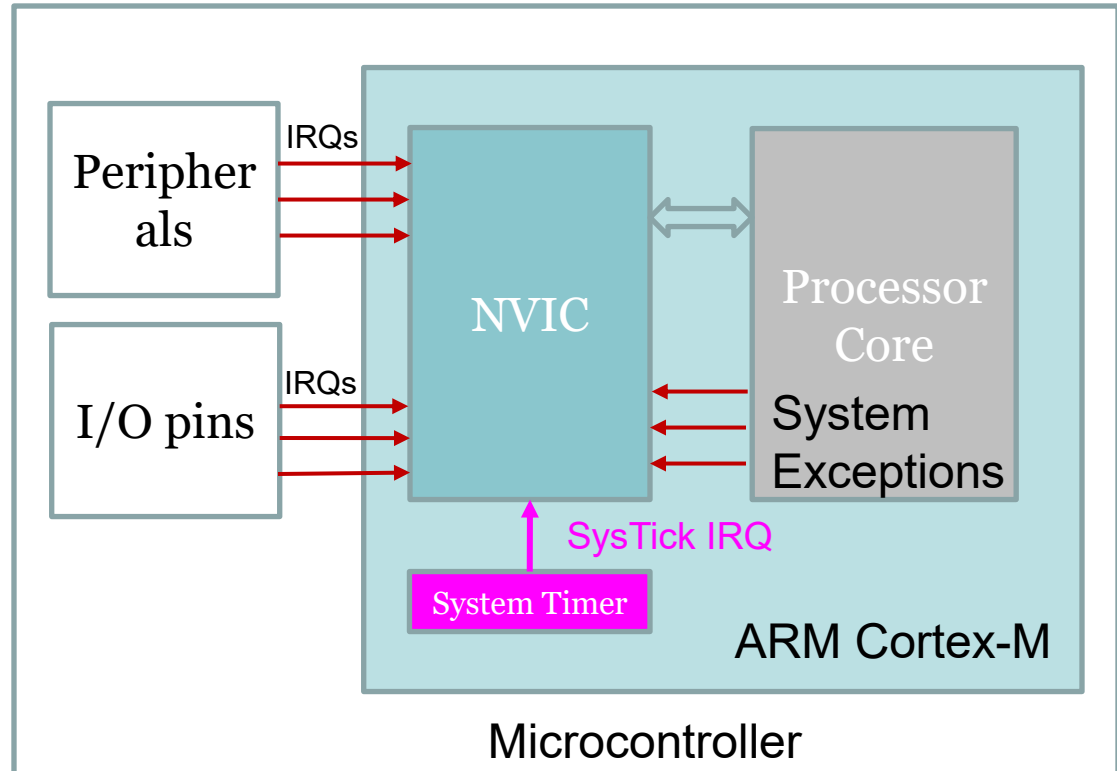
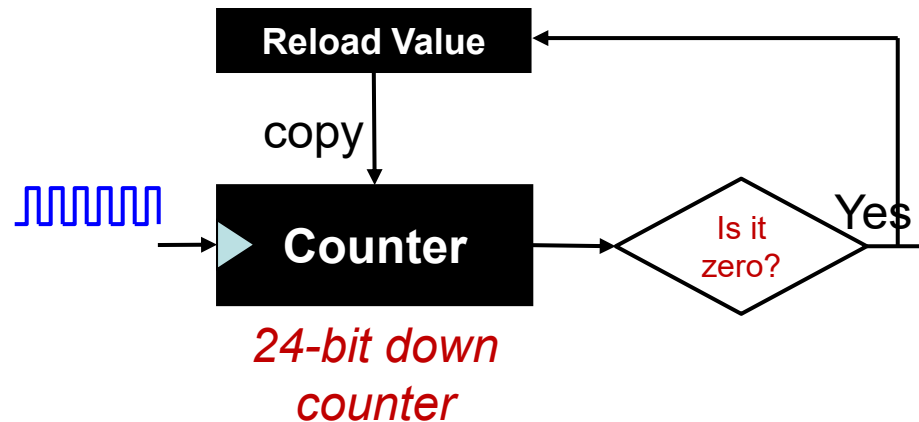
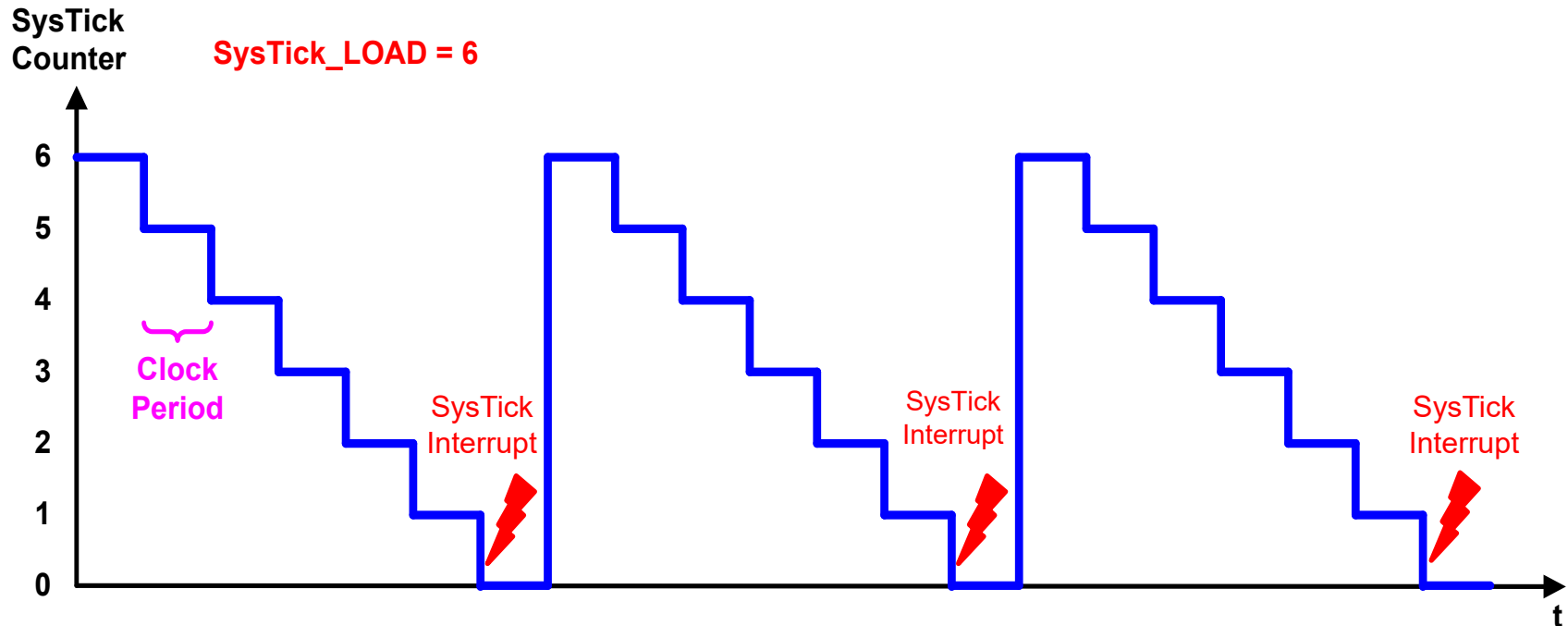


Diagram of System Timer (SysTick)



System Timer



$$\text{SysTick Interrupt Time Period} = (\text{SysTick_LOAD} + 1) \times \text{Clock Period} = 7 \times \text{Clock Period}$$

Diagram of System Timer (SysTick)

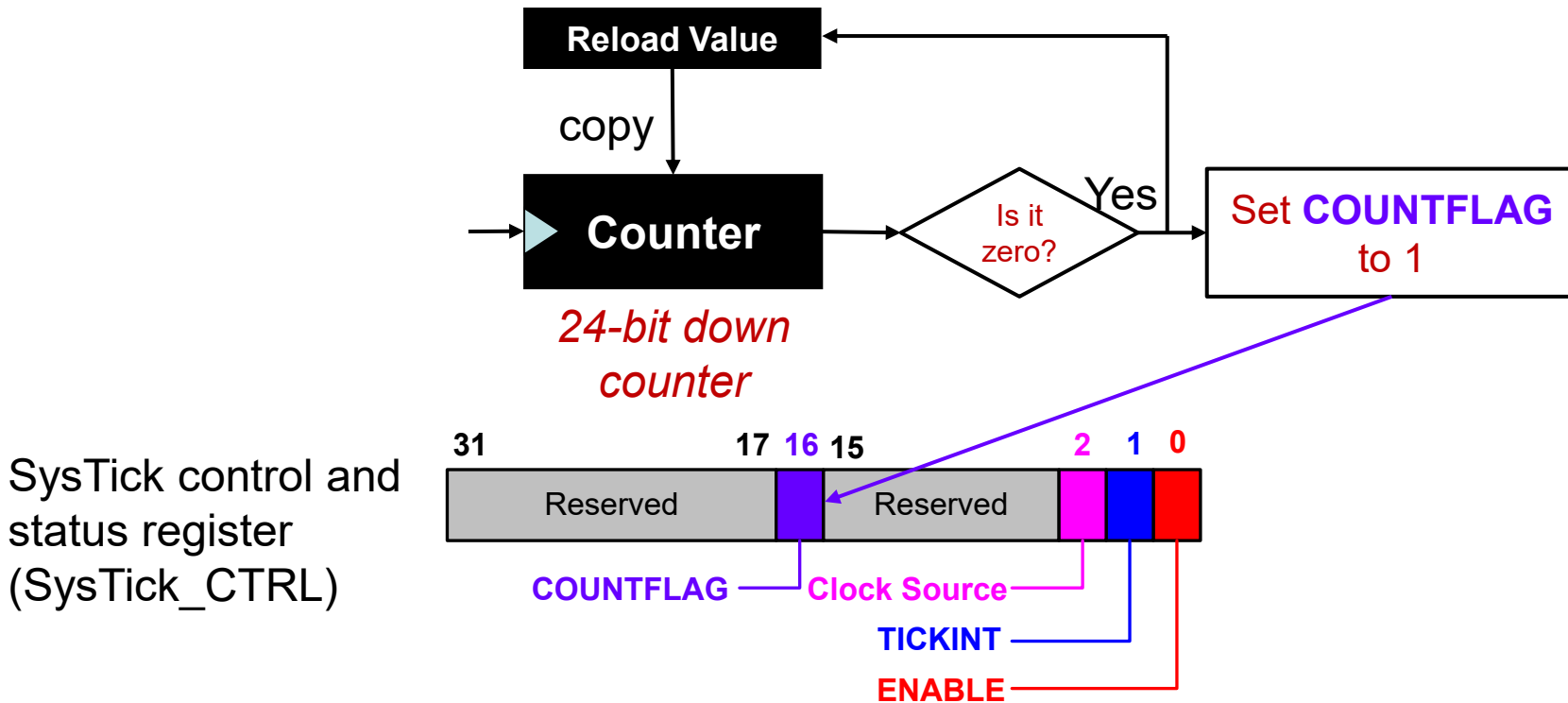
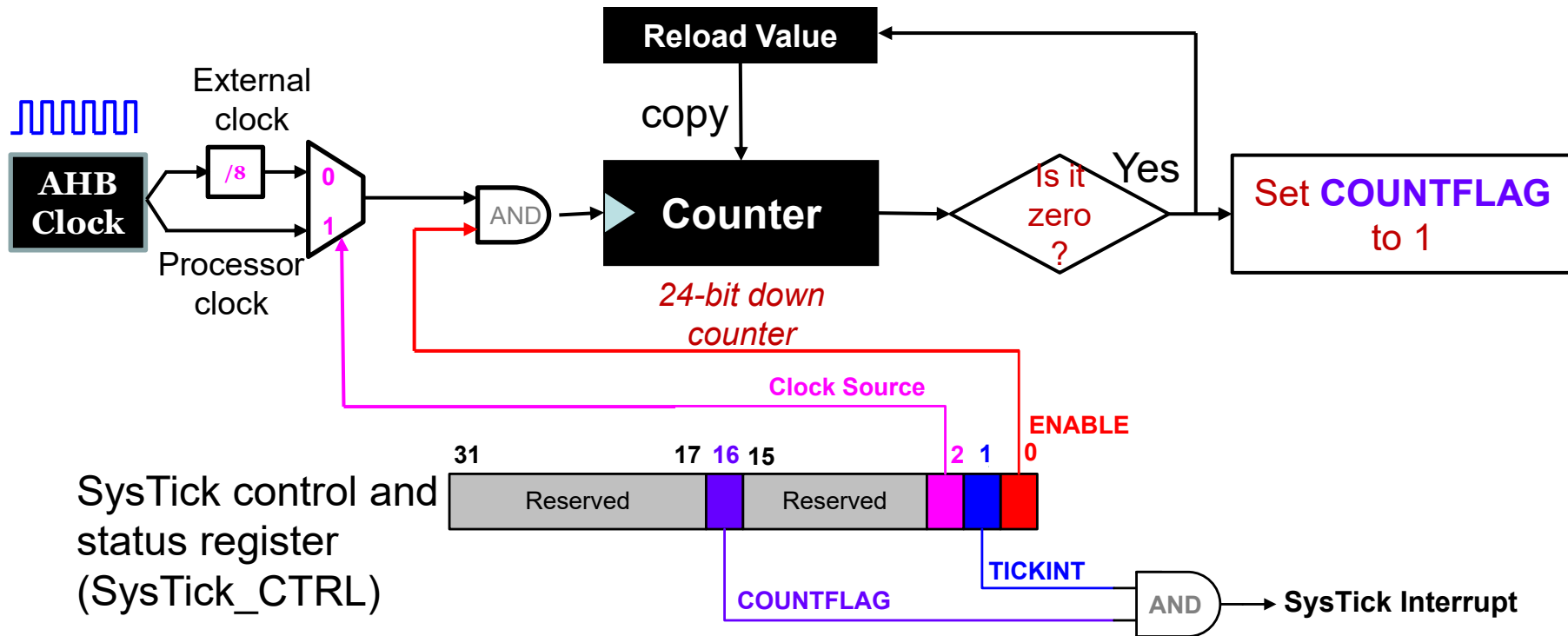
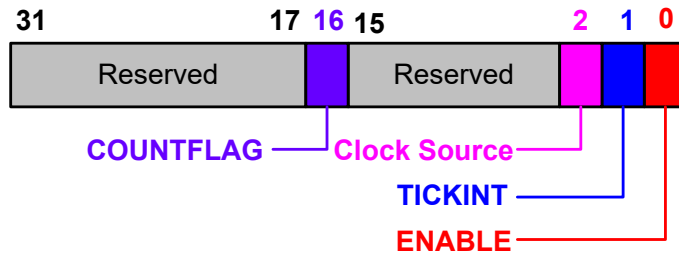


Diagram of System Timer (SysTick)



Registers of System Timer

SysTick control and status register (SysTick_CTRL)



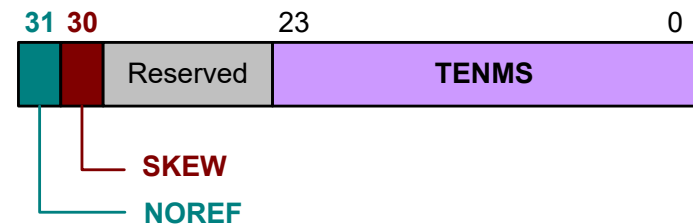
SysTick reload value register (SysTick_LOAD)



SysTick current value register (SysTick_VAL)



SysTick calibration register (SysTick_CALIB)



Registers of System Timer

SysTick reload value register (SysTick_LOAD)



- ▶ 24 bits, maximum value 0x00FF.FFFF (16,777,215)
- ▶ Counter counts down from RELOAD value to 0.
- ▶ Writing RELOAD to 0 disables SysTick, independently of TICKINT
- ▶ Time interval between two SysTick interrupts

$$\text{Interval} = (\text{RELOAD} + 1) \times \text{Source_Clock_Period}$$

- ▶ If 100 clock periods between two SysTick interrupts

$$\text{RELOAD} = 99$$

Registers of System Timer

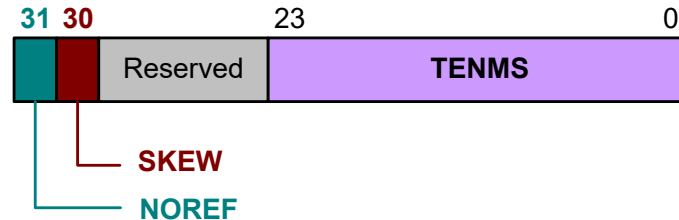
SysTick current value register (SysTick_VAL)



- Reading it returns the current value of the counter
- When it transits from 1 to 0, it generates an interrupt
- Writing to SysTick_VAL clears the counter and COUNTFLAG to zero
 - Cause the counter to reload on the next timer clock
 - But, does not trigger an SysTick interrupt
- It has random value on reset.
 - Always clear it before enabling the timer

Registers of System Timer

SysTick calibration register (SysTick_CALIB)

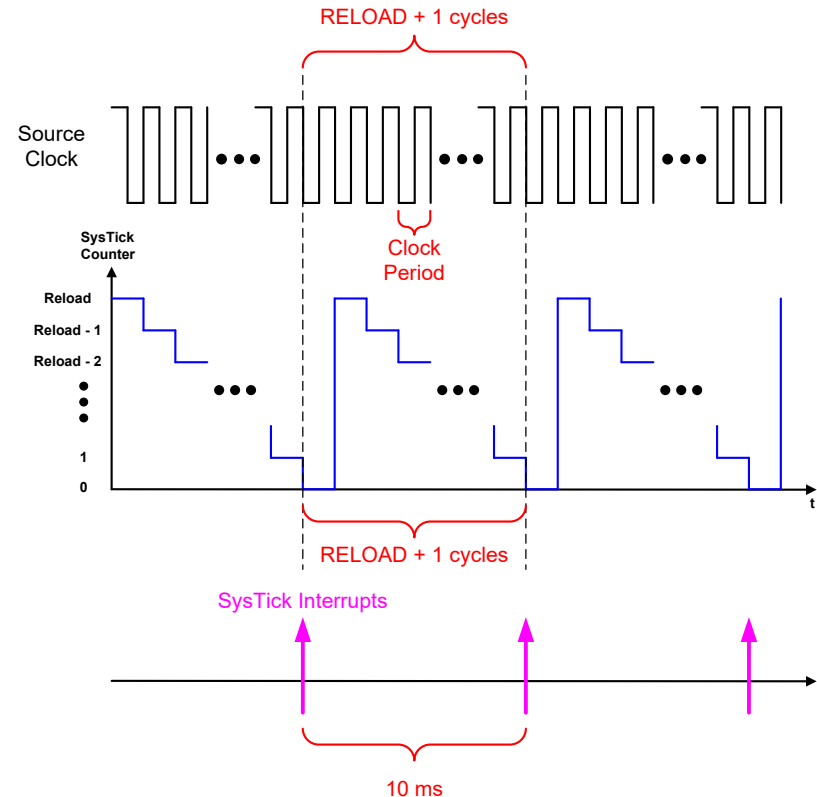


- A read-only register
- TENMS (10 ms) holds the reload value, which will yield a 10ms period
- May not be implemented or may be defined differently by chip designers
- For example, STM32L4 defines this field as the reload value for 1 millisecond, instead of 10 milliseconds

Calculating Reload Value

- Suppose clock source = 80MHz
- Goal: SysTick Interval = 10ms
- What is RELOAD value?

$$\begin{aligned}
 \text{Reload} &= \frac{10 \text{ ms}}{\text{Clock Period}} - 1 \\
 &= 10\text{ms} \times \text{Clock Frequency} - 1 \\
 &= 10\text{ms} \times 80\text{MHz} - 1 \\
 &= 10 \times 10^{-3} \times 80 \times 10^6 - 1 \\
 &= 800000 - 1 \\
 &= 799999
 \end{aligned}$$



Implementing Delay Function

- Purpose: Delay 1 second each time
- Main function
 - Initialize SysTick to generate interrupt every 1 ms
 - Call Delay
- Delay function
 - Set `TimeDelay` to 1000
 - Test if a variable `TimeDelay` is 0
- SysTick Handler
 - Decrease `TimeDelay` each time a SysTick interrupt is generated

Implementing Delay Function

Main function

Initialize SysTick to generate interrupt
every 1 ms
Call Delay

```
TimeDelay DCD 0x00 ; global, volatile variable

__main PROC
    .....
    LDR r0, =1000 ; Interrupt period = 1000 cycles
    BL systick_init
    .....
    LDR r0, =1000 ; Delay 1000 interrupts
    BL delay
    .....
    ENDP
```

Implementing Delay Function

Delay function

Set TimeDelay to 1000

Test if a variable TimeDelay is 0

```
delay PROC
    LDR r1, =TimeDelay
    STR r0, [r1] ; r0 specifies the delay time length
delayloop
    LDR r1, =TimeDelay
    LDR r2, [r1]
    CMP r2, #0
    BGT delayloop ; Busy wait
    BX LR
ENDP
```

Implementing Delay Function

Systick Handler

Decrease TimeDelay each time a
Systick interrupt is generated

```
SysTick_Handler PROC ; SysTick interrupt service routine
    LDR r1, =TimeDelay ; TimeDelay is a global volatile variable
    LDR r2, [r1]
    CMP r2, #0
    SUBGT r2, r2, #1 ; Prevent it from being negative
    STR r2, [r1]
    BX LR
    ENDP
```


Example Code

`systick_init` PROC

`; Disable SysTick`

`LDR r1, =SysTick`

`LDR r2, =0x0`

`STR r2, [r1, #0]`

`;offset of SysTick_CTRL is 0`

`; Set reload register. R0 holds the argument passed to the routine`

`LDR r1, =SysTick`

`SUB r0, r0, #1`

`STR r0, [r1, #4]`

`;offset of SysTick_LOAD is 4`

`; Reset the SysTick counter value`

`LDR r1, =SysTick`

`LDR r2, =0x0`

`STR r2, [r1, #8]`

`;offset of SysTick_VAL is 8`

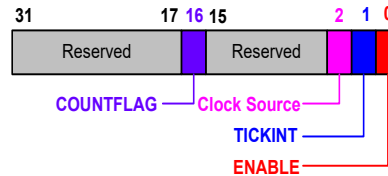
`; Set interrupt priority of SysTick.`

`PUSH {LR, R0}`

`BL config_NVIC_in_C`

`POP {LR, R0}`

SysTick control and status register (SysTick_CTRL)



SysTick reload value register (SysTick_LOAD)



SysTick current value register (SysTick_VAL)



```

LDR r1, =SysTick
LDR r2, [r1, #0]
; Enables SysTick interrupt, 1 = Enable, 0 = Disable
ORR r2, #0x02
STR r2, [r1, #0] ;offset of CTRL is 0
; Select processor clock: 1 = processor clock; 0 = external clock
; Use external 1MHz Clock
LDR r1, =SysTick
LDR r2, [r1, #0]
BIC r2, #0x04
STR r2, [r1, #0] ;offset of CTRL is 0
; Enable SysTick
LDR r1, =SysTick
LDR r2, [r1, #0]
ORR r2, #0x01
STR r2, [r1, #0] ;offset of CTRL is 0
BX LR
ENDP

```

